MCP-Enabled Web Dashboard with Python Tools

Project Proposal & Technical Specification

Version: 1.0 **Date:** July 22, 2026 **Author:** Nicolas **Status:** DRAFT

Category: Technical Report **Classification:** Internal **Audience:** Engineering & Stakeholders

Table of Contents

---

1. Executive Summary

This proposal describes the design and delivery of a production-grade web dashboard that leverages the **Model Context Protocol (MCP)** to surface real-time data and tool outputs from a fleet of connected Python-based MCP servers. As LLM-driven agents become operational infrastructure — not just experimental novelties — engineering teams require purpose-built operator interfaces to observe, control, and audit everything that happens inside an agentic system. This project delivers exactly that: a unified, browser-accessible control plane that sits at the intersection of AI orchestration, developer tooling, and operational visibility.

The dashboard will serve as both an MCP host and an MCP client, establishing persistent connections to multiple Python MCP tool servers — covering file system access, database introspection, and HTTP/API integration — and presenting their outputs through a rich, real-time UI. Operators will be able to inspect live tool call invocations, explore the model's active context window, manage multi-turn agent sessions, and review a tamper-evident audit log, all from a single pane of glass. The underlying architecture is designed for extensibility: new MCP servers can be registered and discovered without modifying application code, enabling the platform to grow alongside the organization's AI tooling surface.

The value proposition is threefold. First, **AI-powered context awareness** — operators gain full visibility into what the model knows, what tools it has called, and what resources it has consumed, eliminating the "black box" opacity that plagues most LLM deployments today. Second, **dynamic tool integration** — the MCP

protocol's standardized client-server interface allows any well-formed Python tool server to be plugged in with minimal friction, dramatically accelerating the time from idea to integrated capability. Third, **a unified operator UI for agentic workflows** — rather than cobbling together terminal output, log files, and ad-hoc scripts, teams get a professional, role-aware interface purpose-built for the demands of production AI systems. This proposal outlines the full technical specification, phased delivery plan, risk profile, and success criteria for the project.

---

2. Project Overview

2.1 What is MCP?

The **Model Context Protocol (MCP)** is an open standard introduced by Anthropic in late 2024 and broadly adopted across the AI industry through 2025 and into 2026. It defines a standardized, transport-agnostic client-server protocol that enables AI models to securely connect to external data sources, services, and tools. Rather than requiring every LLM integration to implement bespoke API adapters, MCP provides a common language that any compliant host, client, or server can speak.

The core architectural concepts in MCP are as follows:

- **MCP Host:** The application (e.g., an AI assistant or, in this case, the web dashboard) that initiates and owns the overall interaction. The host is responsible for orchestrating connections and managing the lifecycle of sessions.
- **MCP Client:** A component embedded within the host that manages the protocol-level connection to a single MCP server — handling handshakes, capability negotiation, and message serialization.
- **MCP Server:** A lightweight, independently deployable service that exposes capabilities (tools, resources, and prompts) to connected clients via the MCP protocol. Servers can be local subprocesses (stdio transport) or remote services (SSE/HTTP transport).

- **Tools:** Callable functions exposed by an MCP server that a model can invoke to perform side-effecting or data-fetching operations (e.g., `read_file`, `query_db`).
- **Resources:** Structured data objects a server makes available for the model to read — files, database schemas, API responses — without requiring an explicit tool call invocation.
- **Prompts:** Parameterized prompt templates that an MCP server can offer, enabling reusable, server-defined interaction patterns.
- **Session:** A stateful, authenticated connection between an MCP client and server, persisted for the duration of an agent interaction.
- **Transport:** The communication channel between client and server — either **stdio** (for local subprocess servers) or **SSE over HTTP** (for remote network servers).

2.2 Project Description

This project delivers a **web-based dashboard that acts as an MCP host and client**, connecting to one or more Python-based MCP servers that expose structured tool interfaces. The dashboard provides operators with a live view of tool outputs, session context, model interactions, and system health — functioning as the command center for any agentic AI workflow running within the organization. The backend is written in Python (FastAPI), the frontend in TypeScript (Next.js 15), and all MCP tool servers are built with the official `mcp` Python SDK.

2.3 Target Users

- **AI Engineers** — developers building and maintaining LLM-based agentic systems who need deep observability into tool execution.
- **ML Ops Teams** — operations personnel responsible for monitoring model health, latency SLAs, and error rates in production.
- **Internal Tooling Teams** — platform engineers who build and register new MCP tool servers for consumption by AI systems.

- **Technical Power Users** — senior practitioners who need to interact with, debug, and steer agentic workflows in real time.

2.4 Problem Statement

As LLM-based agents proliferate in production environments, a critical operational gap has emerged: there is no unified, purpose-built UI to observe, debug, and manage MCP tool calls, context flow, and model outputs in real time. Engineers are forced to piece together insights from terminal logs, database queries, and custom scripts — a slow, error-prone, and unscalable approach. The absence of a dedicated operator interface creates blind spots in system behavior, slows incident response, and makes it difficult to enforce governance and auditability standards. This project fills that gap.

> 💡 **Key Insight**
>
> The MCP-Enabled Dashboard is not just a monitoring tool — it is a **first-class operational primitive** for any organization running AI agents in production. The earlier it is adopted, the more operational maturity the team accrues.

---

3. Goals & Success Criteria

3.1 Primary Goals

1. Build a fully functional, MCP-compliant web dashboard operating as both host and client.
2. Integrate at least **three Python MCP tool servers**: file system, database, and HTTP/API.
3. Enable **real-time tool call visualization** and context inspection within the dashboard UI.
4. Provide **session management** for multi-turn, stateful agent workflows.

5. Achieve **sub-200ms latency** for tool call round-trips under normal load (p95).

3.2 Secondary Goals

6. Modular plugin architecture for adding new MCP tool servers without code changes to the core platform.
7. Role-based access control (RBAC) supporting Admin, Operator, and Viewer roles with tool-level granularity.
8. Append-only, tamper-evident audit logging of all tool invocations with full payload capture.

3.3 Measurable Success Criteria

Criterion	Target	Measurement Method
Tool call round-trip latency	< 200ms at p95	k6 load test against staging environment
MCP servers operational at launch	≥ 3 servers discoverable	Integration test suite pass rate
Security audit critical findings	Zero critical vulnerabilities	Third-party pentest report prior to production release
Dashboard service uptime SLA	≥ 95% monthly uptime	Prometheus uptime metric + Grafana alerting
Developer onboarding time (new tool server)	< 30 minutes end-to-end	Timed walkthrough with a new team member using documentation only
Test suite coverage	≥ 80% line coverage (backend)	Pytest coverage report in CI pipeline

---

4. Technical Architecture

4.1 High-Level Architecture

The system is organized into five distinct layers, each with a well-defined boundary and responsibility. Communication between layers is asynchronous where possible to support real-time streaming and high concurrency. The following describes the architecture from top to bottom:

▲ Layer 1 — Frontend (Browser)

Next.js 15 TypeScript SPA. Renders dashboard panels and receives real-time event streams from the API Gateway via Server-Sent Events (SSE) or WebSocket. Communicates exclusively with the API Gateway — never directly with MCP servers.

▲ Layer 2 — API Gateway (FastAPI)

Python FastAPI application with async Uvicorn runtime. Handles REST endpoints, WebSocket connections, JWT authentication, session management, and WebSocket fan-out to browser clients. Acts as the orchestration hub.

▲ Layer 3 — MCP Client (Embedded in Backend)

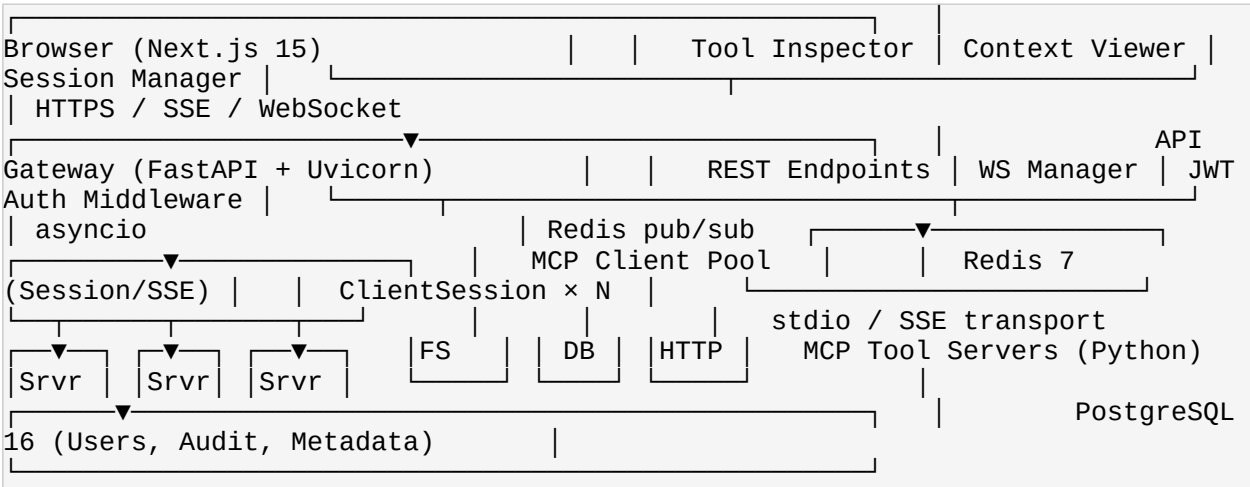
Python `mcp` SDK `ClientSession` instances, one per registered MCP server. Manages capability negotiation, tool dispatch, resource fetching, and connection lifecycle. Connection pool handles multiplexing.

▲ Layer 4 — MCP Server Layer

Multiple independent Python MCP servers (filesystem, database, HTTP). Each runs as a container or subprocess, exposing tools via stdio or SSE transport. Servers are stateless and horizontally scalable.

▲ Layer 5 — Data & State Layer

Redis 7 for session/context state and pub/sub event routing. PostgreSQL 16 for durable user data, RBAC permissions, and the append-only audit log. Optional: pgvector for semantic search over tool results.



4.2 Frontend (Dashboard UI)

The frontend is a single-page application built with **Next.js 15** and **TypeScript**, styled with Tailwind CSS and the shadcn/ui component library. It connects to the backend via REST for initial data loading and via SSE or WebSocket for live event streaming. State is managed using Zustand (local UI state) and React Query (server state with cache invalidation).

The dashboard is organized into six primary panels:

- **Tool Call Inspector:** A live, scrollable feed of MCP tool invocations showing tool name, input parameters, output payload, execution duration, status

(success / error / timeout), and the originating session ID. Supports filtering by server, tool, and status.

- **Context Viewer:** A structured display of the model's current context window — active resources, injected prompts, prior tool results, and token utilization. Updates in real time as the session progresses.
- **Server Registry:** A table of all registered MCP servers with live health status (connected / degraded / offline), tool inventory, transport type, and last-seen timestamp. Supports manual server registration via form.
- **Session Manager:** Create, pause, resume, and terminate agent sessions. Shows active session count, session duration, model assignment, and linked tool servers per session.
- **Model Interaction Panel:** A chat-style interface for sending prompts to the model, observing streamed model responses, and manually injecting tool results or context overrides into the active session.
- **Audit Log:** A searchable, filterable, paginated table of all past tool invocations. Columns: timestamp, user, session ID, server, tool, input hash, output hash, latency (ms), status. Exportable as CSV.

> 📝 **Design Principle**
>
> All dashboard panels are independently scrollable and composable. Operators can open multiple panels simultaneously in a split layout. The UI never blocks on a single tool call — all updates are pushed asynchronously via the SSE stream.

4.3 Backend API (FastAPI)

The backend is a **Python 3.12+ FastAPI application** with full async support, served by Uvicorn with Gunicorn workers in production. It is the single point of entry for all frontend requests and the sole orchestrator of MCP client connections.

Authentication: JWT-based, with short-lived access tokens (15 minutes) and long-lived refresh tokens (7 days) stored in HTTP-only cookies. OAuth2 Password flow for v1; OAuth2 SSO (Google, GitHub) planned for v2.

Core API Endpoints:

Method	Endpoint	Description
`GET`	`/api/servers`	List all registered MCP servers with status and tool inventory
`POST`	`/api/servers`	Register a new MCP server (Admin only)
`POST`	`/api/sessions`	Create a new agent session with model and server assignments
`GET`	`/api/sessions/{id}`	Retrieve session state, context, and tool call history
`POST`	`/api/sessions/{id}/invoke`	Invoke a named tool on the session's connected MCP server
`GET`	`/api/sessions/{id}/stream`	SSE stream for all real-time events within a session
`DELETE`	`/api/sessions/{id}`	Terminate a session and release MCP client resources
`GET`	`/api/audit`	Paginated, filterable audit log query
`GET`	`/api/health`	System health check — all server connections, DB, Redis

Long-running tool executions are dispatched to a background task queue (asyncio `TaskGroup` for in-process; Celery with Redis broker as optional heavy-load extension). OpenAPI/Swagger documentation is auto-generated at `/docs`.

4.4 MCP Client Integration

The MCP client layer uses the official **`mcp` Python SDK** to establish and maintain `ClientSession` instances — one per registered MCP server. Each session handles the full MCP handshake lifecycle: initialization, capability negotiation, tool schema exchange, and graceful shutdown.

Key implementation details:

- **Transport support:** Both `stdio` (for co-located subprocess servers) and `SSE over HTTP` (for remote network servers) are supported and configurable per server.
- **Connection pool:** A connection manager maintains a pool of active `ClientSession` objects, keyed by server ID. Sessions are lazily initialized on first use and eagerly re-established on failure.
- **Automatic reconnection:** Exponential backoff with jitter (max 30s) for transient connection failures. Dead servers are marked degraded and polled for recovery every 60 seconds.
- **Tool schema caching:** Tool schemas (name, description, input JSON Schema) are fetched during capability negotiation and cached in Redis with a 5-minute TTL, reducing per-call overhead.
- **Observability hooks:** Every tool dispatch emits an OpenTelemetry span with server ID, tool name, invocation timestamp, and result status — enabling end-to-end distributed tracing from UI click to tool response.

4.5 Python MCP Tool Servers

Three core MCP servers will be built and shipped with the initial release. Each is an independent Python 3.12 application using the `mcp` SDK's server primitives, containerized with Docker, and deployable via the shared Kubernetes Helm chart.

📁 Server 1 — File System Server (`mcp-server-filesystem`)

Purpose: Provides sandboxed, controlled access to a designated workspace

directory on the host or in a shared volume.

Exposed Tools:

- `read_file(path)` — Read the contents of a file (text or binary, base64-encoded)
- `write_file(path, content)` — Write or overwrite a file within the sandbox root
- `list_directory(path)` — Return directory listing with file metadata (size, modified, type)
- `search_files(pattern, root)` — Glob-based file search within the sandbox
- `get_file_info(path)` — Return detailed metadata for a single file or directory

Security: All path arguments are canonicalized and validated against a configurable `WORKSPACE_ROOT` environment variable. Symlink following is disabled. The server process runs in a container with a read-only root filesystem (write access only to the workspace mount).

📄 Server 2 — Database Server (`mcp-server-database`)

Purpose: Exposes read-only SQL query access and schema introspection for connected PostgreSQL or SQLite databases.

Exposed Tools:

- `query_db(sql, params)` — Execute a parameterized, read-only SQL query; returns rows as JSON
- `list_tables(schema)` — Return all table names in a given schema
- `describe_table(table_name)` — Return column names, types, nullability, and constraints

- `get_schema()` — Return the full database schema as a structured object

Security: Connects via SQLAlchemy with a read-only database role (no INSERT, UPDATE, DELETE, DROP privileges). All queries use parameterized bindings — no string interpolation. Result sets are limited to a configurable maximum row count (default: 500 rows) to prevent runaway queries.

🌐 Server 3 — HTTP/API Server (`mcp-server-http`)

Purpose: Enables the agent to make controlled outbound HTTP requests to pre-approved external APIs and web services.

Exposed Tools:

- `http_get(url, headers)` — Perform an HTTP GET request; returns status, headers, and body
- `http_post(url, body, headers)` — Perform an HTTP POST with a JSON body
- `fetch_json(url)` — Fetch and parse a JSON endpoint; validates Content-Type
- `parse_response(raw, format)` — Post-process a raw response (JSON extraction, HTML stripping, truncation)

Security: All target URLs are checked against a configurable domain allowlist before the request is dispatched. Rate limiting is enforced per tool per session (default: 10 req/min). Response bodies exceeding 512 KB are automatically summarized before being returned to the model context.

⚠ Security Note — Tool Server Isolation

Each MCP tool server runs in its own container with the principle of least privilege enforced at the OS level: dropped Linux capabilities, seccomp profiles, no host network access (except for `mcp-server-http` via a controlled egress proxy), and non-root user execution. Tool servers must *never* share a container

with the API gateway.

4.6 Data & State Layer

Redis 7 serves as the primary in-memory data store for all ephemeral and real-time data:

- Session context objects (tool call history, active resource cache, current prompt state) stored as JSON hashes with a configurable TTL (default: 24 hours).
- MCP tool schema cache with a 5-minute TTL per server.
- Pub/sub channels for routing real-time tool call events from the backend to connected SSE streams.
- Rate limiting counters for `mcp-server-http` tool calls (sliding window via Redis sorted sets).

PostgreSQL 16 provides durable, relational storage for all persistent data:

- **Users table:** ID, email, hashed password (bcrypt), roles, created/updated timestamps.
- **RBAC tables:** Roles, permissions, role-permission mapping, user-role mapping.
- **Audit log table:** Append-only; columns include: `id` (UUID), `session_id`, `user_id`, `server_id`, `tool_name`, `input_json`, `output_json`, `latency_ms`, `status`, `timestamp`, `prev_hash` (SHA-256 of prior row for hash chaining).
- **Server registry table:** Registered MCP servers with configuration, transport type, and metadata.

Vector Store (Stretch): pgvector extension on the existing PostgreSQL instance (or a dedicated Pinecone index) for semantic search over historical tool results and context snapshots, enabling the Natural Language Tool Discovery stretch feature.

4.7 Security Architecture

Security is a first-class concern across all layers of the system. The following controls are applied:

Control Area	Implementation
MCP Server Authentication	Shared secrets (HMAC) for stdio servers; mutual TLS (mTLS) for remote SSE servers
API Authentication	JWT access tokens (RS256, 15-min expiry) + HTTP-only refresh token cookies
Authorization	RBAC with three roles: Admin (full access), Operator (invoke tools, manage sessions), Viewer (read-only)
Tool-Level Permissions	Each tool can be individually gated per RBAC role in the server registry configuration
Input Validation	Pydantic models on all API request bodies; JSON Schema validation on all MCP tool inputs before dispatch
Output Sanitization	All tool output rendered in the UI is HTML-escaped; binary payloads are never injected into the DOM
Container Isolation	Tool servers run with dropped Linux capabilities, seccomp profiles, and read-only root filesystems
Secrets Management	All secrets via environment variables in development; HashiCorp Vault integration in production
Audit Log Integrity	Append-only audit log with SHA-256 hash chaining across rows; log deletion requires Admin + 2FA confirmation
Transport Encryption	TLS 1.3 enforced on all external endpoints; internal cluster communication encrypted via Kubernetes network policies

4.8 Deployment Architecture

The system is designed for **container-first, environment-parity deployment** across development, staging, and production:

- **Local Development:** Docker Compose orchestrates all services (frontend dev server, FastAPI, three MCP servers, Redis, PostgreSQL) with hot-reload enabled for Python and TypeScript.
- **Production:** Kubernetes (k8s) with a shared Helm chart. Each service runs as its own Deployment with resource requests/limits defined. MCP servers are deployed as separate Deployments with HorizontalPodAutoscaler (HPA) rules.
- **CI/CD:** GitHub Actions pipeline with stages: lint (Ruff, ESLint), test (Pytest, Playwright), build (Docker multi-stage), push (GHCR), deploy (Helm upgrade with rollback on failure).
- **Environments:** `dev` (branch deploys, ephemeral), `staging` (mirror of production, persistent), `production` (blue/green deployment with canary releases).
- **Observability:** OpenTelemetry SDK instrumented in FastAPI and MCP client layer; traces exported to Jaeger. Prometheus metrics scraped from all services. Grafana dashboards for latency, error rates, tool call volume, and session counts.

---

5. Project Milestones & Timeline

The project follows a **phased, MVP-first delivery strategy**. Milestones M1 through M3 establish the foundational platform. M4 and M5 complete the full feature set. M6 adds observability and governance. M7 hardens the system for production release. Stretch features from Section 8 are deferred to a post-MVP backlog and scheduled after M7 based on team capacity and stakeholder priority.

Milestone	Description	Key Deliverables	Target	Owner
M1: Foundation	Project setup, architecture finalized, development environment running	Monorepo, Docker Compose stack, CI pipeline, Architecture Decision Records (ADRs)	Week 2	Nicolas
M2: MCP Core	MCP client integrated into backend; first tool server live and callable	`mcp-server-filesystem` operational; API endpoints for tool invocation; MCP client pool	Week 5	Backend Lead
M3: Dashboard MVP	Basic frontend operational with real-time tool call feed	Next.js dashboard shell; Tool Call Inspector panel; Server Registry panel; SSE integration	Week 8	Frontend Lead
M4: Full Tool Suite	All three MCP servers complete; session management live in UI	`mcp-server-database`, `mcp-server-http`; Session Manager panel; Context Viewer panel	Week 11	Backend Lead
M5: Auth & RBAC	Authentication, user management, and role-based access control implemented	JWT auth flow; RBAC data model; login / register / token refresh UI; tool-level permission gates	Week 13	Nicolas
M6: Audit & Observability	Audit logging, metrics collection, and distributed	Audit Log panel with CSV export; Prometheus + Grafana	Week 15	Platform Lead

Milestone	Description	Key Deliverables	Target	Owner
	tracing live	setup; OpenTelemetry traces in Jaeger		
M7: Hardening & Launch	Security review, performance testing, full documentation, production deploy	Pentest report (zero critical findings); k6 load test results (<200ms p95); user & developer docs; production Helm deploy	Week 18	Nicolas

🕑 **Phased Approach Note**

Each milestone has a hard gate: the team does not advance to the next milestone until the current milestone's acceptance criteria are met and sign-off is received from Nicolas. Scope additions after M1 must go through a formal change request to protect the Week 18 launch date.

---

6. Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend	Next.js, TypeScript, Tailwind CSS, shadcn/ui	Next.js 15, TS 5.x	Dashboard UI, real-time panels, SSE client
Frontend State	Zustand, React Query	Latest stable	Local UI state; server state + cache management

Layer	Technology	Version	Purpose
Backend API	FastAPI, Uvicorn, Gunicorn	FastAPI 0.115+	REST + WebSocket API gateway, auth middleware
Backend Language	Python	3.12+	All backend and MCP server code
MCP Client	mcp Python SDK, asyncio	Latest stable	MCP protocol client, tool dispatch, session mgmt
MCP Servers	Python, mcp SDK, SQLAlchemy, HTTPX	SQLAlchemy 2.x, HTTPX 0.27+	File system, database, HTTP tool execution
Session State	Redis	7.x	Context storage, pub/sub, rate limiting counters
Database	PostgreSQL	16.x	Users, RBAC, audit log, server registry
ORM / Query	SQLAlchemy (async)	2.x	Database access layer for FastAPI and DB server
Authentication	PyJWT, bcrypt, OAuth2 (future)	Latest stable	Token generation, password hashing, auth flows
Containerization	Docker, Docker Compose, Kubernetes	Docker 26+, k8s 1.30+	Local dev orchestration; production deployment
Helm	Helm	3.x	Kubernetes package management and deployment
CI/CD	GitHub Actions	—	Lint, test, build, push, deploy pipeline
Observability	OpenTelemetry, Prometheus, Grafana, Jaeger	Latest stable	Distributed tracing, metrics collection, dashboards
Testing — Unit	Pytest, pytest-asyncio	Pytest 8.x	Backend unit and integration tests

Layer	Technology	Version	Purpose
Testing — E2E	Playwright	Latest stable	Browser-based end-to-end UI testing
Testing — Load	k6	Latest stable	Performance and latency SLA validation
Secrets	Environment variables (dev), HashiCorp Vault (prod)	Vault 1.17+	Secret storage and injection

---

7. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation Strategy
MCP specification evolves mid-project, breaking SDK compatibility	Medium	High	Pin the `mcp` SDK to a specific version at project start. Introduce an internal abstraction layer over the SDK so spec changes require localized updates only. Assign one engineer to monitor the official MCP changelog weekly.
Tool server sandbox escape — malicious or malformed tool input breaks container isolation	Low	Critical	Container-level isolation with seccomp profiles, dropped capabilities, and read-only root filesystems. All tool inputs validated against JSON Schema

Risk	Likelihood	Impact	Mitigation Strategy
			before dispatch. Penetration test specifically targeting sandbox boundaries before M7 launch.
Tool call round-trip latency exceeds 200ms p95 SLA under load	Medium	Medium	Profile tool call performance from M2 onwards using OpenTelemetry spans. Use connection pooling for MCP client sessions. Cache tool schemas in Redis. Implement async parallel dispatch for independent tool calls. Load test with k6 at M6 and M7.
Scope creep from stretch features pulled into core milestones	High	Medium	Strict milestone gates enforced by Nicolas. Stretch features are formally tracked in a separate backlog, not the active sprint. Any scope addition after M1 sign-off requires written change request and impact assessment.
Key engineer unavailability disrupts delivery of critical milestones	Low	High	Maintain comprehensive Architecture Decision Records (ADRs) and inline code documentation from day one. Cross-train at least two engineers on each

Risk	Likelihood	Impact	Mitigation Strategy
			subsystem. No single-engineer knowledge silos permitted on the critical path.
PostgreSQL audit log grows unbounded under high tool call volume	Medium	Low	Implement table partitioning by month from the start. Define a retention policy (e.g., 90 days hot, archive to S3 Glacier thereafter). Alert when table size exceeds configurable threshold.

---

8. Stretch Features (Post-MVP Backlog)

The following features are explicitly out of scope for the v1.0 release but are documented here to inform future architectural decisions. All stretch features are tracked in the project backlog and will be prioritized after the Week 18 production launch.

- **Natural Language Tool Discovery:** Allow operators to describe what they need in plain English, and the dashboard recommends the most appropriate available MCP tool — powered by an embedded LLM with semantic search over tool schemas stored in pgvector. Reduces the cognitive load of browsing a large tool inventory.
- **Visual Workflow Builder (MCP Orchestrator):** A drag-and-drop canvas for chaining MCP tool calls into multi-step, conditional workflows. Supports

branching logic, loops, variable passing between tool calls, and export of workflow definitions as YAML or JSON for version-controlled reuse.

- **Multi-Model Support:** Connect multiple LLM providers (OpenAI GPT-4o, Anthropic Claude, local Ollama instances) within the same dashboard. Route sessions to different models based on task type. Side-by-side comparison mode for evaluating model + tool combinations on identical inputs.
- **MCP Server Marketplace / Plugin Registry:** A curated registry of community-built Python MCP tool servers. Operators can browse, install, configure, and activate new servers directly from the dashboard UI without touching infrastructure.
- **Collaborative Sessions:** Real-time multi-user collaboration on a single agent session. Multiple operators can simultaneously observe, inject context, or invoke tools, with optimistic concurrency control and conflict resolution for simultaneous edits.
- **Replay & Time-Travel Debugging:** Record entire agent sessions as structured event logs and replay them step-by-step. Operators can rewind to any prior state, inspect the context at that point, and branch from any past event to test alternative tool call sequences — a DVR for AI agents.
- **Mobile Companion App:** A React Native application for monitoring active session health, receiving push notifications on long-running tool completions and errors, and reviewing the audit log on mobile devices.
- **Self-Hosted MCP Server Generator:** A UI wizard that scaffolds a new Python MCP server from a JSON Schema definition of desired tools, generates complete boilerplate code with tests, and automatically registers the new server with the dashboard — reducing a new tool server from hours to minutes.

---

9. Open Questions

The following questions require resolution from stakeholders before or during project execution. They are tracked as blocking items in the project backlog and will be reviewed at the M1 architecture sign-off meeting.

9. **LLM Provider for v1 Model Interaction Panel:** Which LLM provider will be integrated in the initial release — OpenAI GPT-4o, Anthropic Claude 3.5+, or a locally-hosted model via Ollama? The choice affects API key management, latency characteristics, cost model, and data residency.
10. **Filesystem Server Deployment Model:** Should `mcp-server-filesystem` run as a shared sidecar container (one per node) or as a dedicated per-user-session ephemeral container? The latter provides better isolation but significantly increases infrastructure cost and cold-start latency.
11. **Target Deployment Environment:** Is the production target a self-managed Kubernetes cluster, a managed cloud Kubernetes service (GKE, EKS, AKS), or a simpler single-server Docker Compose deployment? This determines infrastructure automation scope at M1.
12. **SIEM Integration from Day One:** Should the audit log support real-time streaming to external SIEM platforms (Splunk, Datadog, Elastic) at v1 launch, or is CSV export sufficient for the initial release with SIEM forwarding deferred to v1.1?
13. **Auth System Requirements for v1:** Is email/password authentication (with JWT) sufficient for the v1 launch, or is OAuth2 SSO via Google or GitHub required from day one? SSO requires additional implementation effort and a registered OAuth2 application per provider.
14. **Compliance & Data Residency Requirements:** Do any data residency, regulatory, or compliance requirements apply to this system (SOC 2 Type II, HIPAA, GDPR, FedRAMP)? The answer materially affects the database configuration, encryption requirements, audit log retention policy, and deployment region selection.

10. Appendix

A. Glossary

Term	Definition
MCP	Model Context Protocol — an open standard introduced by Anthropic (late 2024) defining a standardized client-server protocol for connecting AI models to external data sources, tools, and services.
Host	The top-level application that owns an AI interaction session and coordinates one or more MCP clients. In this project, the FastAPI backend + Next.js dashboard together constitute the MCP host.
Client	A protocol-level component (typically a `ClientSession` instance in the Python SDK) that maintains a single connection to one MCP server, handles handshakes, and dispatches tool calls on behalf of the host.
Server	An independently deployable service that exposes tools, resources, and/or prompts to connected MCP clients via the MCP protocol. In this project, the three Python servers (`mcp-server-filesystem`, `mcp-server-database`, `mcp-server-http`) are MCP servers.
Tool	A callable function exposed by an MCP server. Tools accept structured inputs (validated against a JSON Schema) and return structured outputs. They may have side effects (e.g., writing a file, making an HTTP request).
Resource	A structured data object (file content, database schema, API response) that an MCP server makes available for the model to read without requiring an explicit tool call. Resources are

Term	Definition
	identified by a URI.
Prompt	A parameterized prompt template offered by an MCP server. Prompts allow servers to define reusable, structured interaction patterns that the host can instantiate with specific variable values.
Session	A stateful, authenticated connection between an MCP client and server, persisted across multiple tool calls within a single agent interaction lifecycle.
Transport (stdio)	A transport mode in which the MCP client communicates with a local MCP server process via standard input/output streams. Used for co-located, subprocess-based servers.
Transport (SSE)	A transport mode in which the MCP client communicates with a remote MCP server via HTTP, using Server-Sent Events for server-to-client streaming. Used for network-accessible, independently deployed servers.
RBAC	Role-Based Access Control — an authorization model in which permissions are assigned to roles, and users are assigned to roles, rather than permissions being assigned to users directly.
SSE	Server-Sent Events — a unidirectional HTTP-based streaming protocol used in this project to push real-time tool call events from the FastAPI backend to the Next.js frontend without requiring a full WebSocket connection.

B. References

- **MCP Specification:** Model Context Protocol official documentation and specification — `modelcontextprotocol.io`
- **MCP Python SDK:** Official Python SDK for building MCP hosts, clients, and servers — `github.com/modelcontextprotocol/python-sdk`

- **FastAPI Documentation:** FastAPI framework reference and async patterns — `fastapi.tiangolo.com`
- **Next.js Documentation:** Next.js 15 app router and React Server Components — `nextjs.org/docs`
- **shadcn/ui:** Accessible, composable React component library — `ui.shadcn.com`
- **SQLAlchemy 2.x Async Docs:** Async ORM patterns for Python — `docs.sqlalchemy.org`
- **OpenTelemetry Python:** Instrumentation SDK for distributed tracing — `opentelemetry.io/docs/languages/python`
- **HashiCorp Vault:** Secrets management platform — `developer.hashicorp.com/vault`

C. Document History

Version	Date	Author	Status	Summary of Changes
1.0	July 22, 2026	Nicolas	Draft	Initial document — full technical specification, architecture, milestones, risks, and stretch features. Prepared for stakeholder review.

📄 **Document Status**

This document is version **1.0 DRAFT** as of **July 22, 2026**, authored by **Nicolas**. It is pending stakeholder review and sign-off prior to project kickoff. All open questions in Section 9 must be resolved before the M1 architecture sign-off at

the end of Week 2. Please direct feedback and approvals to Nicolas.